

BusinessSAAS

Swag API Documentation Guide

How to set up, write, and regenerate API docs using swaggo/swag

For: Developers & Future Team Members | June 2026

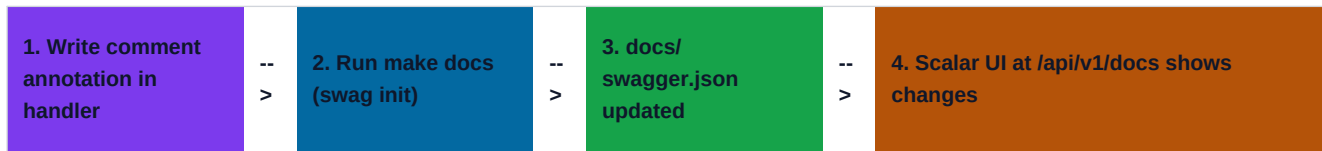
Stack: Go / Fiber v3 / swaggo v1.16 / Scalar UI

Contents		Page
Section 1	How Swag Works	2
Section 2	File Structure in This Project	2
Section 3	Daily Workflow — 4 Steps	3
Section 4	Anatomy of an Annotation Block	3
Section 5	Every Annotation Tag Explained	4
Section 6	The 4 Endpoint Patterns	5
Section 7	Annotating Struct Fields	6
Section 8	Common Mistakes to Avoid	7
Section 9	Quick Reference Card	8

Section 1 — How Swag Works

Understanding the pipeline from code comment to live documentation

Swag (swaggo/swag) is a CLI tool that reads special comment blocks written above Go handler functions and generates an OpenAPI 3.0 specification file (swagger.json). That JSON file is then served by the backend at runtime and rendered in the browser by Scalar UI — giving every developer an interactive, always-up-to-date API reference without any manual maintenance.



Section 2 — File Structure in This Project

Where the documentation files live and what each one does

Project layout (docs-related files)

```
backend/  
  cmd/server/main.go      <- Global @title, @version, @security annotations  
  docs/  
    swagger.json          <- Generated OpenAPI 3.0 spec (do not edit manually)  
    docs.go               <- Embeds swagger.json into the binary at build time  
  internal/  
    auth/handler.go       <- Example: fully annotated handler (read this first)  
    task/handler.go       <- Annotated handler  
    crm/leads/handler.go  <- Annotated handler  
    ... (all other handlers) <- Add annotations when you add new endpoints  
  Makefile                <- make docs-install / make docs / make dev
```

File	What it does	Edit it?
docs/swagger.json	The generated OpenAPI spec. Scalar UI reads this to render the interactive docs.	NEVER — auto-generated by swag
docs/docs.go	Go package that embeds swagger.json at compile time. The binary is self-contained.	NEVER — auto-generated
cmd/server/main.go	Contains the global API-level annotations: title, version, security scheme, and the route that serves /api/v1/docs and /api/v1/docs/openapi.json.	Only for global settings
internal/*/handler.go	Each handler function gets its own annotation block directly above it. This is where 99% of your documentation work happens.	YES — add per endpoint

Documentation URLs (development only):

URL	What you see
http://localhost:8080/api/v1/docs	Scalar UI — interactive API browser with purple dark theme
http://localhost:8080/api/v1/docs/openapi.json	Raw OpenAPI 3.0 JSON — import into Postman, Insomnia, or any API tool

Section 3 — Daily Workflow

The 4 steps every developer follows when adding a new endpoint

Rule: Every new handler function needs an annotation block. No annotation = the endpoint does not appear in the docs. Existing endpoints already have annotations — follow the same pattern.

Write the annotation block

- 1 Add the comment block directly above the handler function in the relevant handler.go file. Copy the pattern from an existing handler in the same file (or see Section 6 for the 4 patterns).

Run make docs

- 2 This runs swag init, which scans all handler files and regenerates docs/swagger.json. You will see the count of routes processed in the terminal output.

Start the server and verify

- 3 Run make dev (Air hot-reload) then open <http://localhost:8080/api/v1/docs> in a browser. Find your new endpoint in the sidebar and confirm the summary, parameters, and response shapes all look correct.

Commit both files

- 4 Commit the edited handler.go file AND the regenerated docs/swagger.json together in the same commit. This keeps the spec always in sync with the code in Git history.

Make commands reference:

Command	When to run	What it does
<code>make docs-install</code>	Once per machine, or after upgrading Go	Installs the swag CLI tool globally (go install)
<code>make docs</code>	After every annotation change	Runs swag init and regenerates docs/swagger.json
<code>make dev</code>	During development	Starts the server with Air hot-reload on port 8080
<code>make docs-open</code>	Quick shortcut	Opens http://localhost:8080/api/v1/docs in the default browser

Section 4 — Anatomy of an Annotation Block

What every line means in a complete annotation

Below is a real example from this project (auth/handler.go). Every handler annotation follows this exact structure. The comment must start with `// FuncName godoc` and be separated from the func keyword by the `//` tags.

internal/auth/handler.go — complete annotation example

```
// Login godoc
//
// @Summary      Log in
// @Description  Authenticates with email + password.
// @Description  On success: returns access_token in body + sets httpOnly refresh cookie.
// @Tags         Auth
// @Accept       json
// @Produce      json
// @Param        body body LoginRequest true "Login credentials"
// @Success      200 {object} LoginResponseData "Access token issued"
// @Failure      400 {object} response.Error "INVALID_BODY"
// @Failure      401 {object} response.Error "INVALID_CREDENTIALS"
// @Failure      429 {object} response.Error "RATE_LIMITED"
// @Router       /auth/login [post]
func (h *Handler) Login(c fiber.Ctx) error {
```

Line	Tag	What it does
1	// FuncName godoc	Required opening line. FuncName must match the actual Go function name exactly.
2	//	Empty comment line. Separates the opening from the tags. Do not omit this.
3	@Summary	One-line title shown in the sidebar and as the endpoint heading. Keep it short.
4-5	@Description	Full description shown in the detail panel. Can span multiple lines by repeating @Description. Use this for error codes, side-effects, and notes.
6	@Tags	Groups the endpoint in the sidebar. Must match exactly the tag names defined in main.go.
7	@Accept	Request body format. Always json for this project.
8	@Produce	Response body format. Always json for this project.
9	@Param	Describes one request input. See Section 5 for the full @Param syntax.
10-13	@Success / @Failure	Describes response shapes. @Success for 2xx, @Failure for 4xx/5xx. Each takes: status code, type hint, schema name, and optional description.
14	@Router	The URL path and HTTP method. Path params use {name} curly-brace syntax.

Section 5 — Every Annotation Tag Explained

Complete reference for all tags used in this project

Tag	Syntax	Example	Notes
@Summary	@Summary Short text	@Summary Create a lead	One line only. Shown in sidebar and as the endpoint title.
@Description	@Description Text	@Description Requires crm.leads.create permission.	Repeat on multiple lines for longer descriptions. Use for permission requirements and error code explanations.
@Tags	@Tags TagName	@Tags CRM - Leads	Must exactly match a tag defined in main.go @tag.name. Controls which sidebar group the endpoint appears in.
@Accept	@Accept format	@Accept json	Always json in this project. Tells Scalar what format the request body is in.
@Produce	@Produce format	@Produce json	Always json in this project. Tells Scalar what format responses are in.
@Security	@Security SchemeName	@Security BearerAuth	Marks the endpoint as requiring auth. Add this to all non-public endpoints. BearerAuth is defined once in main.go.
@Param (path)	@Param name path string true "desc"	@Param orgId path string true "Organization UUID"	For URL path variables like /:orgId. Always required=true for path params.
@Param (query)	@Param name query type false "desc"	@Param status query string false "Filter by status"	For ?key=value query string parameters. Use false for optional, true for required.
@Param (body)	@Param name body TypeName true "desc"	@Param body body CreateLeadRequest true "Lead data"	For JSON request bodies. TypeName must be a Go struct visible to swag. The name is always body by convention.
@Success	@Success code {type} TypeName "desc"	@Success 201 {object} LeadResponseData "Lead created"	Use {object} for structs, {array} for slices. Common codes: 200 OK, 201 Created, 204 No Content.
@Failure	@Failure code {object} TypeName "desc"	@Failure 403 {object} response.Error "FORBIDDEN"	Add one @Failure line per error case. Always include 401, 403 for protected endpoints. Put the error code in the description string for quick reference.
@Router	@Router /path [method]	@Router /organizations/{org Id}/crm/leads [post]	The URL path relative to BasePath (/api/v1). Path params use {curly} braces. Method is lowercase: get, post, patch, delete.

Section 6 — The 4 Endpoint Patterns

Copy the right pattern based on what your endpoint needs

Every endpoint in this project is one of these 4 patterns. Find the pattern that matches your new endpoint and copy it as a starting template.

PATTERN 1 — Public (no authentication required)

Example: POST /auth/login

```
// Login godoc
//
// @Summary      Log in
// @Description  Authenticates with email + password.
// @Description  On success: returns access_token body + sets httpOnly cookie.
// @Tags         Auth
// @Accept       json
// @Produce      json
// @Param        body body LoginRequest true "Login credentials"
// @Success      200  {object} LoginResponseData
// @Failure      400  {object} response.Error "INVALID_BODY"
// @Failure      401  {object} response.Error "INVALID_CREDENTIALS"
// @Failure      429  {object} response.Error "RATE_LIMITED"
// @Router       /auth/login [post]
func (h *Handler) Login(c fiber.Ctx) error {
```

Note: No @Security tag. Public endpoints are open. Rate limited at the middleware level.

PATTERN 2 — Auth required (JWT bearer token, no org scope)

Example: GET /me

```
// GetProfile godoc
//
// @Summary      Get my profile
// @Description  Returns the authenticated user's full profile.
// @Tags         User
// @Produce      json
// @Security     BearerAuth
// @Success      200  {object} ProfileResponseData
// @Failure      401  {object} response.Error "UNAUTHORIZED"
// @Failure      500  {object} response.Error
// @Router       /me [get]
func (h *Handler) GetProfile(c fiber.Ctx) error {
```

Note: @Security BearerAuth added. No orgId param because this endpoint has no org scope.

PATTERN 3 — Org-scoped (JWT + orgId path param, no explicit permission)

Example: GET /organizations/{orgId}/tasks

```
// List godoc
//
// @Summary      List tasks
// @Description  Returns tasks for this org. Requires tasks.view permission.
// @Tags         Tasks
// @Produce      json
// @Security      BearerAuth
// @Param        orgId path string true "Organization UUID"
// @Param        status query string false "Filter: todo|in_progress|done|cancelled"
// @Param        limit query integer false "Page size (default 50, max 200)"
// @Param        offset query integer false "Pagination offset"
// @Success      200 {object} TaskListResponse
// @Failure      401 {object} response.Error "UNAUTHORIZED"
// @Failure      403 {object} response.Error "FORBIDDEN"
// @Router       /organizations/{orgId}/tasks [get]
func (h *Handler) List(c fiber.Ctx) error {
```

Note: orgId path param always first. Query params listed below it. 403 included because RequirePermission middleware fires before the handler.

PATTERN 4 — Org-scoped with body + permission (most common CRM pattern)

Example: POST /organizations/{orgId}/crm/leads

```
// Create godoc
//
// @Summary      Create lead
// @Description  Creates a new CRM lead. Requires crm.leads.create permission.
// @Tags         CRM - Leads
// @Accept       json
// @Produce      json
// @Security      BearerAuth
// @Param        orgId path string true "Organization UUID"
// @Param        body body CreateLeadRequest true "Lead details"
// @Success      201 {object} LeadResponseData "Lead created"
// @Failure      400 {object} response.Error "INVALID_BODY"
// @Failure      401 {object} response.Error "UNAUTHORIZED"
// @Failure      403 {object} response.Error "FORBIDDEN - missing crm.leads.create"
// @Failure      500 {object} response.Error
// @Router       /organizations/{orgId}/crm/leads [post]
func (h *Handler) Create(c fiber.Ctx) error {
```

Note: @Accept json added because there is a request body. @Param body always after all path params. 201 for creation, not 200.

Section 7 — Annotating Struct Fields

How to make the Scalar UI show realistic examples for request and response bodies

When swag reads a struct referenced in `@Param` body or `@Success`, it generates a schema from the struct's fields. By default the examples are empty. Adding struct-level field comments and example tags makes the Scalar UI show realistic sample values, which is essential for junior developers trying to understand what to send.

internal/crm/leads/model.go — struct with example tags

```
// CreateLeadRequest defines the request body for POST /crm/leads.
type CreateLeadRequest struct {
    // Required. Lead's first name.
    FirstName string `json:"first_name" example:"John"`

    // Optional. Lead's last name.
    LastName string `json:"last_name" example:"Smith"`

    // Optional. Must be a valid email address.
    Email string `json:"email" example:"john@acme.com"`

    // Optional. Phone number including country code.
    Phone string `json:"phone" example:"+1-555-0100"`

    // Lead acquisition channel. Values: website, referral, cold-call, event.
    Source string `json:"source" example:"website"`
}
```

Rule	Explanation
Add a // comment above each field	The comment becomes the field description in the Scalar UI schema panel.
Add example:"value" in the struct tag	Swag reads this and shows it as the example value in the request body sample.
Use validate:"required" for required fields	Swag picks up the validate tag and marks the field as required in the schema.
For enums, list valid values in the comment	For example: // Values: new, contacted, qualified. Swag will show these as the allowed options.
Never add example tags to sensitive fields	Do not add example tags to password, token, or secret fields.

Response wrapper types:

Many handlers return `fiber.Map{}` rather than a typed struct. Swag cannot infer the shape of `fiber.Map`, so the Scalar UI shows an empty schema. To fix this, define a simple wrapper struct at the bottom of the handler.go file and reference it in the `@Success` tag. These structs are only for documentation and are never used in runtime code.

Add at the bottom of internal/crm/leads/handler.go

```
// Swagger response wrapper types
// Only used for documentation schema generation -- not used in runtime code.

// LeadResponseData is the data field returned by single-lead endpoints.
type LeadResponseData struct {
    Lead *Lead `json:"lead"`
}

// LeadListResponseData is the data field returned by the list endpoint.
type LeadListResponseData struct {
    Leads []Lead `json:"leads"`
    Total int    `json:"total"`
}
```

Section 8 — Common Mistakes to Avoid

Errors that cause swag init to silently produce wrong output or fail



Forgetting the empty `//` line after `// FuncName godoc`

The blank comment line between the function name and the first tag is required. If it is missing, the block at all and the endpoint will disappear from the docs.



`@Router` path uses `:param` instead of `{param}`

Fiber route definitions use `:orgId` syntax, but the `@Router` annotation must match the OpenAPI standard. Example: `@Router /organizations/{orgId}/ta`



Referencing a struct type that swag cannot find

If the struct in `@Param` body or `@Success` is in a different package that swag cannot find, it will show an empty schema. Run `make docs` with `--parseDependency` flag (also see Section 7).



Using `fiber.Map{} in @Success without a wrapper struct`

swag cannot infer the shape of `fiber.Map`. Always define a typed response struct (see Section 7).



`@Tags` value does not match a tag defined in `main.go`

If the tag name in the handler does not exactly match one of the `@tag.name` tags in `main.go`, the endpoint will appear under an ungrouped section. Check `main.go` for the correct tag name.



Editing `docs/swagger.json` manually

Any manual edits to `swagger.json` will be overwritten the next time `make docs` is run. Changes must be made through annotations in the handler files.



Not committing `swagger.json` after `make docs`

If you add an annotation but forget to commit the regenerated `swagger.json`, the generated docs will have an outdated spec. Always commit the handler file and `swagger.json`.



Putting `@Router` before `@Summary`

While swag does not strictly require tag order, the conventional order is: `@Accept`, `@Produce`, `@Security`, `@Param` lines, `@Success` lines, `@Failure` lines, and `@Router`. Putting `@Router` first makes code harder to read.

Section 9 — Quick Reference Card

Everything you need on one page — pin this to your desk

Minimal annotation template (copy and fill in):

TEMPLATE — copy this, remove comments, fill in the values

```
// FunctionName godoc
//
// @Summary      <one line description>
// @Description  <longer description. Add permission requirement here.>
// @Tags         <match exactly one tag name from main.go>
// @Accept       json                                <- add only if there is a request body
// @Produce      json
// @Security     BearerAuth                          <- remove for public endpoints
// @Param        orgId path string true "Organization UUID" <- remove if not org-scoped
// @Param        body body RequestTypeName true "Description" <- remove if GET/DELETE
// @Success      200 {object} ResponseTypeName "Success message"
// @Failure      400 {object} response.Error "INVALID_BODY"
// @Failure      401 {object} response.Error "UNAUTHORIZED"
// @Failure      403 {object} response.Error "FORBIDDEN"
// @Router       /your/path/{orgId}/resource [method]
func (h *Handler) FunctionName(c fiber.Ctx) error {
```

HTTP Status	Use for
200	Successful GET, PATCH, POST (non-creation)
201	Successful resource creation (POST)
204	Successful DELETE or logout (no body)
400	Bad request, validation error, missing fields
401	Missing or invalid JWT token
403	Authenticated but lacks permission
404	Resource not found or wrong org
409	Conflict — duplicate resource
429	Rate limited (auth endpoints)
500	Unexpected server error

Tag Name in @Tags	Module
System	Health, hello
Auth	Login, signup, refresh, OAuth
User	Profile, avatar
Organizations	Create, list, switch
RBAC	Roles, permissions, members
Security	Sessions, login events
Tasks	Task CRUD
CRM - Contacts	Platform contacts
CRM - Companies	Platform companies
CRM - Engagement	Notes, activities, emails, timeline
CRM - Leads	Lead CRUD + convert
CRM - Pipeline	Pipelines + stages
CRM - Deals	Deal CRUD + Kanban
CRM - Reports	Analytics endpoints

Checklist before submitting a PR with a new endpoint:

[] Annotation block starts with // FuncName godoc followed by an empty // line

[] @Summary is one line and clearly describes what the endpoint does

- [] @Tags matches exactly one tag name from the list in main.go
- [] @Security BearerAuth is present (unless this is a deliberately public endpoint)
- [] orgId path @Param is listed first if the endpoint is org-scoped
- [] @Param body is present with the correct request struct name (if request body exists)
- [] @Success uses 201 for creation, 200 for reads/updates, 204 for deletes
- [] @Failure includes 401 and 403 for all protected endpoints
- [] @Router path uses {curly} braces, not :colon, for path parameters
- [] make docs has been run and docs/swagger.json is included in the commit
- [] The new endpoint appears correctly in Scalar UI at <http://localhost:8080/api/v1/docs>